

Vibe Coding

Build by Intention, Not by Syntax



The hottest new programming language is English.

— Andrej Karpathy, 2025



Powerful LLMs



System Prompts



Execution Envs

What Is Vibe Coding?

A new paradigm introduced by Andrej Karpathy in early 2025

Definition



A style of software development where the primary work of writing code is **shifted to an AI tool**. You describe intent in natural language and the AI generates the code.

Commonly Associated With



- ▶ Rapid prototyping & MVPs
- ▶ Solo founders & non-engineers
- ▶ Building full apps from plain English
- ▶ AI handles boilerplate & infrastructure

Core Idea: "Almost forgetting that the code exists" — Idea → Prompt → Result

THE VIBE CODING FLOW



Benefits of Vibe Coding

Why teams and solo founders are adopting this approach



Rapid Validation

Build MVP and test assumptions with real users in hours, not weeks



Lower Upfront Cost

No large dev team needed — a single founder can ship a product



Tighter Feedback Loops

Founder, designer, PM work in real-time; iterate in small slices



Better Product Intuition

Visual evolution reveals UX friction that documents miss



Iterate & Measure

Ship changes iteratively, test with cohorts, measure real impact

Three Pillars of Vibe Coding

The foundation that makes AI-driven development possible



Powerful LLMs

Large language models like GPT, Claude, and Gemini that understand code semantics, architecture patterns, and can generate production-quality code from natural language descriptions.



Sophisticated System Prompts

Carefully crafted system prompts that define coding standards, tech stacks, security constraints, and architectural guidelines — turning generic LLMs into specialized dev assistants.



Integrated Execution Environments

Platforms like Lovable, Bolt, GitHub Copilot and Cursor that provide end-to-end environments: code generation, live preview, code review, authentication, and one-click deployment.

Two Categories of Vibe Coding Tools

No-code app builders for non-developers vs. AI-powered IDEs for professional developers

Non-Developers — No-Code App Builders

Lovable · [React + Supabase](#)

Prompt-to-full-stack app (React + Supabase). Raised \$330M at \$6.6B valuation; \$200M ARR in <18 months.

Bolt.new · [Next.js / Vite](#)

Browser-based AI builder; generates complete web apps from one prompt. No local setup needed.

Replit Agent · [End-to-end builder](#)

Conversational agent: writes, runs, and deploys code inside Replit's cloud IDE. Widely used for prototyping.

Google AI Studio · [Gemini prototyping](#)

Prototype Gemini-powered apps with a visual playground. Supports Gemini 2.0 Flash & Pro with one-click export.

Developers — AI-Powered IDEs

Cursor · [AI-native IDE](#)

VS Code fork with agentic AI: background agents, BugBot PR reviews, multi-file edits. Valued at \$29.3B (Nov 2025).

GitHub Copilot · [Agentic coding](#)

Full agentic coding in VS Code, JetBrains, Xcode. Agent mode + MCP support; assigns GitHub issues autonomously.

Google Antigravity · [Agent-first IDE](#)

VS Code fork powered by Gemini 3 Pro. Multi-agent Manager view; agents plan, code & test via browser. Free preview.

Amazon Q Developer · [AWS-integrated](#)

Agentic coding (top SWE-Bench score), MCP support, code reviews in GitHub. Covers full AWS dev lifecycle.

Pro Code Tools vs. No-Code Platforms

Choosing the right approach for your team and project

Aspect	Pro Code Tools	Prompt-Based No-Code
Examples	GitHub Copilot, Cursor, Amazon CodeWhisperer	Lovable, Bolt.new, v0, Replit
Suitable For	Experienced developers speeding up coding	Entrepreneurs, founders, PMs, non-coders
Interface	IDE extensions (VS Code, JetBrains)	Web-based chat with live preview
Primary Use	Inline code suggestions within existing IDEs	End-to-end app generation from prompts
Code Ownership	You maintain the codebase actively	AI generates complete, deployable applications
Output Quality	Inline suggestions, code completions	Full-stack applications (frontend + backend)
Deployment	Manual or DevOps	1-Click, Self-hosting

What No Code Platforms Deliver

Full-stack capabilities from a single natural-language prompt



Full-Stack Generation

Frontend + backend code from a single prompt



Built-in Auth & RBAC

User login, role-based access control out of the box



Database Integration

Supabase/PostgreSQL with auto-generated schema



Payment Processing

Stripe integration, subscription management



Email & Notifications

Built-in email integration and alert systems



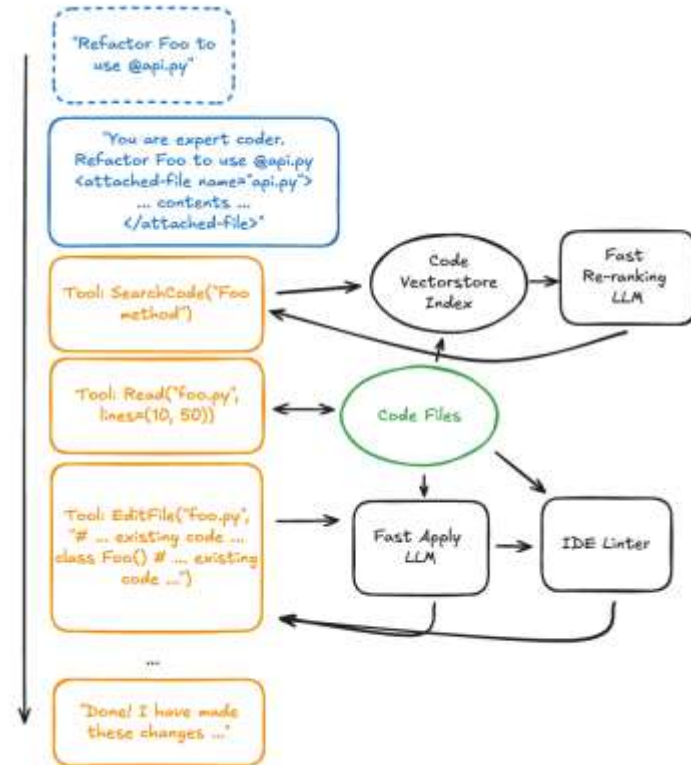
1-Click Deployment

Instant hosting with HTTPS, CORS, env configs

How Agentic Software Works

Unlike traditional AI assistants, agentic AI systems demonstrate autonomous decision-making, adaptability, and goal-oriented behavior.

They handle complex, multi-step development tasks with minimal human intervention through a network of collaborative agents.





Best Practices for Vibe Coding

1. Project Setup & Planning

Strategic foundations for AI-assisted development

Define Clear Intent Before Prompting

Don't just say "Build me a dashboard." Describe functionality, audience, and integrations explicitly.

"Build a responsive sales analytics dashboard with Supabase, Google login, and region/date filters."

Start Small and Iterate

Begin with a minimal feature set (MVP). AI performs best on focused prompts — expand iteratively.

Recommended flow: Auth → Data CRUD → Filters → Charts → Styling

Understand the Tool's Output Type

Lovable outputs React + Supabase full-stack code. Bolt outputs full web apps (Next.js or Vite).

Knowing the output type guides your hosting, framework, and CI/CD decisions.

2. Prompting Best Practices

Be explicit, iterative, and behavior-focused

✗ Vague Prompt

```
"Build me a dashboard."
```

✓ Explicit Prompt

```
"Build a responsive web dashboard  
for sales analytics. Use  
Supabase, Google login,  
TailwindCSS with dark mode.  
Include region & date filters."
```

Start Small & Iterate

Begin with MVP: Auth → CRUD → Filters → Charts → Styling

Describe Behaviors

"Add a line chart showing daily signups over 30 days" not just "Add a chart"

Use Iterative Commands

"Add nav sidebar with icons" → "Replace table with cards" → "Make it responsive"

3. Code Review & Security Essentials

AI-generated code requires disciplined review and hardening



Reviewing Generated Code

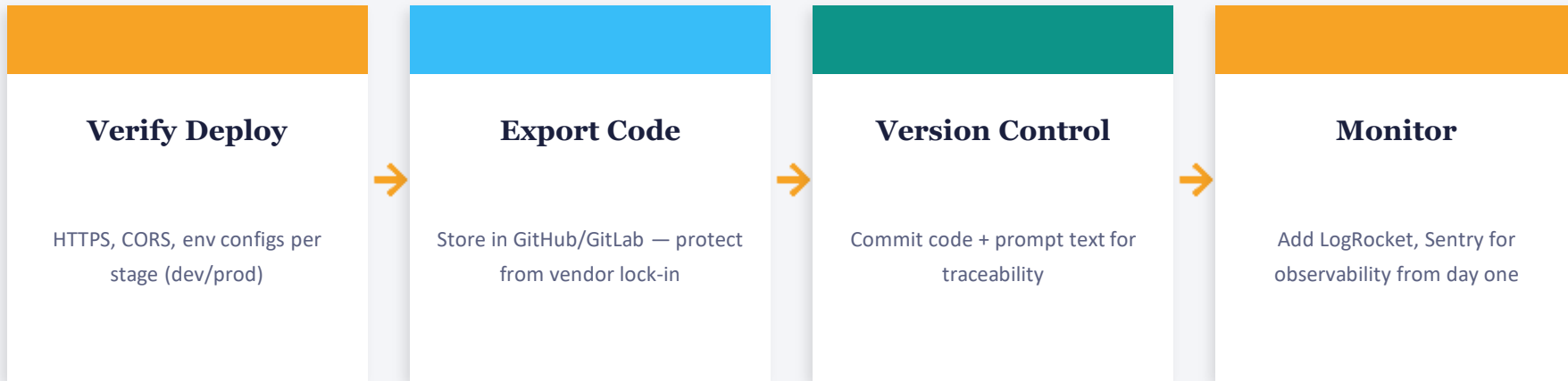
- ☐ Check for unused dependencies & redundant imports
- ☐ Refactor inline styles into proper modules
- ☐ Add missing error handling & validation
- ☐ Scan for hardcoded credentials & API keys
- ☐ Use Cursor/Copilot to refactor after each round
- ☐ Add tests with Vitest, Jest, or Playwright

Security & Data Handling

- ☐ Validate data at both frontend & backend
- ☐ Move all secrets to .env / platform secrets
- ☐ Never rely solely on AI-generated validation
- ☐ Configure Row-Level Security (RLS) rules
- ☐ Verify secure Auth policies for read/write
- ☐ Ensure no public APIs expose sensitive data

4. Deployment & Maintenance

From generation to production-ready deployment



Recommended Tool Chain

Lovable (generate base app) → Cursor (refine code) → GitHub Copilot (add logic/tests) → Vercel / Netlify (deploy)

5. Mindset & Quality Assurance

The human developer remains essential in the AI-assisted loop



AI = Collaborator, Not Replacement

You own design thinking, security, and testing. AI handles implementation speed, not architectural correctness.



Continuous Cross-Platform Testing

Test across browsers, devices, and user roles. AI outputs can miss responsive or accessibility nuances.



Keep Human Skills Sharp

Knowing React, APIs, and deployment basics makes your prompts 10× more effective.



Document Your Prompt Workflow

Keep a 'prompt log' alongside code. Helps future developers understand how key parts were generated.

Challenges of Vibe Coding

Critical risks every team must address



Hallucinated Logic

AI generates code that looks correct but fails under edge cases

→ Treat AI output as untrusted input



Architecture Drift

Without constraints, AI mixes patterns and breaks layering

→ Specify architecture in every prompt



Prompt Quality Gap

Vague prompts → wrong libraries, security gaps

→ Structured prompts with acceptance criteria



Skill Atrophy Risk

Reduced debugging ability & blind trust in outputs

→ Require devs to explain generated code



False Productivity

Fast generation ≠ correct software; time shifts to debugging

→ Track defect rate, not just speed

More Challenges to Navigate

Testing, security, and debugging in AI-generated codebases



Testing Is More Critical

AI may mirror its own flawed assumptions in tests

→ Design test strategy manually



Security & Compliance

Insecure defaults, mishandled secrets, PII violations

→ State security rules in every prompt



Harder Debugging

Weaker mental model when you didn't write the code

→ Enforce code walkthroughs post-gen

Key Takeaways

- Vibe Coding shifts development from syntax to intent — describe what you want, AI writes the code
- Three pillars enable it: powerful LLMs, sophisticated prompts, and integrated execution environments
- Prompt quality directly determines output quality — be explicit, iterative, and behavior-focused
- Always review AI-generated code: check for security gaps, unused deps, and missing error handling
- AI is a collaborator, not a replacement — you still own architecture, security, and testing
- Track defect rate and rework, not just delivery speed — fast generation ≠ correct software
- Keep your human skills sharp — React, API, and deployment knowledge makes prompts 10× better